Development of the “BKIT-COM” Multi-Interface IoT Communication Protocol

---

1. Overview and Objectives

In modern Internet of Things (IoT) applications, efficient and secure data communication among heterogeneous devices is paramount. Devices in an IoT ecosystem often employ different hardware interfaces—such as UART, SPI, or I2C—and must exchange data reliably despite these lower-level differences. Furthermore, cross-platform data encoding strategies, like **Protocol Buffers (Protobuf)**, reduce overhead and simplify message handling on resource-constrained devices.

In this assignment, students will design and implement a custom multi-layered communication protocol, **BKIT-COM**, which serves as a unifying framework for IoT devices that communicate over multiple hardware interfaces. The protocol will incorporate a standardized header, checksums (CRC), optional security features (e.g., keys or flags), and a message serialization layer based on **Protobuf** with the **nanopb** library. Through this project, students will gain experience in protocol design, embedded systems programming, and cross-platform data serialization.

Specific Objectives

1. **Explore Layered Protocol Architecture**
    - **Objective:** Apply core concepts of layered design (akin to the OSI model but adapted for embedded systems) by defining distinct layers for hardware interfaces, message framing (header, CRC), and high-level application functions (send/receive).
    - **Why It Matters:** Layering promotes modularity, readability, and maintainability, ensuring that changes in one layer (e.g., switching from UART to SPI) do not require rewriting higher-layer code.
2. **Implement Robust Messaging with CRC and Headers**
    - **Objective:** Design a packet structure that includes a header with fields such as packet length, CRC, and optional flags or keys. Implement error detection mechanisms (CRC checks), and demonstrate the protocol’s response to invalid packets.
    - **Why It Matters:** Reliability is crucial in IoT environments where data corruption can occur. A proper header and CRC verify message integrity and provide the basis for error handling or retransmission policies.
3. **Incorporate Protobuf and nanopb for Efficient Serialization**
    - **Objective:** Define a simple Protobuf schema (e.g., sensor data), generate code using nanopb, and integrate encoding/decoding routines into the message processing layer.

    - **Why It Matters:** Protobuf is widely used for lightweight and language-agnostic data serialization. nanopb is specifically designed for memory-constrained embedded devices, making it ideal for IoT systems.
4. **Develop Application-Level APIs for Send/Receive**
    - **Objective:** Expose user-friendly functions (e.g., `bkit_send_message()`, `bkit_receive_message()`) that hide lower-level implementation details from the application.
    - **Why It Matters:** Abstracting the communication layer allows application developers to focus on higher-level logic (like sensor data processing) rather than worrying about the specifics of CRC calculation or interface handling.
5. **Demonstrate Multi-Interface Compatibility**
    - **Objective:** Show that the BKIT-COM protocol works consistently over at least one (preferably more) hardware interface(s): UART, SPI, or I2C. Students should be able to swap out drivers with minimal changes to the rest of the stack.
    - **Why It Matters:** Real-world IoT deployments must communicate over various serial protocols. Designing a unified protocol that remains consistent regardless of the hardware interface is both a key academic exercise and an industry-relevant skill.
6. **Foster Clear Documentation and Testing Methodology**
    - **Objective:** Produce design documents, inline code comments, and test evidence (e.g., console logs, hex dumps) that validate correct operation and highlight error-handling procedures.
    - **Why It Matters:** Engineering best practices require thorough documentation and testing. Effective communication of design decisions—along with robust tests—ensures long-term maintainability and reliability.

Expected Learning Outcomes

Upon completing this assignment, students will be able to:

- **Analyze and apply** the principles of layered protocol design to embedded systems.
- **Design and implement** data framing (headers, CRC) within a custom protocol.
- **Integrate** Protobuf (with nanopb) into an embedded project for efficient message serialization.
- **Develop** interface-agnostic APIs for sending and receiving IoT data across different hardware interfaces.
- **Assess and demonstrate** protocol robustness by testing message integrity, reliability, and portability.

---

2. Detailed Requirements

In this section, you will find an in-depth explanation of each requirement for designing the BKIT-COM protocol, with an emphasis on how to structure your solution and what to document. The goal is to ensure that all critical elements—ranging from low-level hardware drivers to high-level message APIs—are clearly defined, consistent, and robust.

2.1 Protocol Layering

A protocol stack is divided into distinct layers, each responsible for a well-defined set of tasks. For the BKIT-COM protocol, at least three layers are recommended:

1. **Hardware Interface Layer**
   - Focuses on low-level operations (e.g., sending/receiving raw bytes via UART, SPI, or I2C).
   - Exports a minimal set of APIs (e.g., `hw_init()`, `hw_send_byte()`, `hw_receive_byte()`).
2. **Message Processing Layer**
   - Handles the framing of data (header inclusion, CRC checks, optional security fields).
   - Provides encode/decode functionality for higher layers.
   - Interprets raw bytes received from the hardware interface and produces validated messages.
3. **Application Layer**
   - Implements the high-level send/receive routines (e.g., `bkit_send_message()`, `bkit_receive_message()`).
   - Integrates with Protobuf (via nanopb) to serialize and deserialize structured data (sensor readings, commands, etc.).

> **Key Takeaway:** This layered approach ensures that **changing the underlying hardware interface** does not disrupt the higher-level message processing or application logic. Each layer must be **isolated** and **well-documented**.

2.2 Hardware Interface Layer

The Hardware Interface Layer is responsible for **abstracting away the physical communication details**. Depending on your chosen interface (UART, SPI, I2C, or others), you will design low-level drivers to send and receive bytes. Below are specific guidelines:

1. **Interface Choice**
   - You **must** choose at least one interface to implement (UART, SPI, or I2C). If possible, demonstrate two or more to illustrate the protocol's flexibility.
   - Each interface driver should present a **consistent set of functions** to the higher layer (e.g., `init`, `write`, `read`) even though the underlying code differs.
2. **Initialization**
   - Include an `hw_init()` function to configure baud rate (UART), clock polarity/phase (SPI), or address settings (I2C).
   - Document any **hardware registers** or **vendor-specific libraries** used.
3. **Data Transmission and Reception**
   - Implement blocking or non-blocking (interrupt/DMA-based) functions for sending/receiving bytes.
   - **Recommendation:** Start with blocking I/O for simplicity, then optionally expand to non-blocking for better performance.

- Handle **error conditions** like bus collision (I2C), overrun errors (UART), or invalid clock phase/polarity settings (SPI).

4. **Driver Interface**
   - Provide a well-defined API, for instance:

```
bool                                        hw_init(void);
bool                hw_send_byte(uint8_t                data);
bool hw_receive_byte(uint8_t* data);
```

   - Higher layers should **not** directly manipulate hardware registers; they must call these driver functions.

Example Reference: If using STM32 microcontrollers, you might rely on the STM32 HAL (Hardware Abstraction Layer) for UART or SPI. If using Arduino, you might use `Serial.begin()` (UART) or the `Wire` library (I2C). Clearly document any library calls.

2.3 Message Processing Layer

The Message Processing Layer is where **packetization** and **data integrity** checks are performed. This layer is interface-agnostic: it should function the same way regardless of whether the underlying interface is UART, SPI, or I2C.

1. **Header Structure**
   - Every outgoing message must have a **common header** that at minimum contains:
     1. **Packet Length**: Specifies the number of bytes of the payload (or the total message size minus the header).
     2. **CRC**: A checksum for verifying message integrity (CRC-8, CRC-16, CRC-32, etc.).
     3. **Flags or Key** (optional): Could indicate encryption use, message priority, or other metadata.
   - Clearly define the **byte order** (endianness) and any special markers (e.g., a start-of-packet byte) in your design document.
2. **CRC Computation**
   - Select an appropriate polynomial and bit length (e.g., CRC-16-CCITT or CRC-32).
   - Implement or use an existing **CRC library**. Provide an example of how you compute the CRC in code and how you validate it upon reception.
   - If a packet fails the CRC check, your receive logic should **discard** or trigger an error-handling routine.
3. **Encoding & Decoding**
   - **Encoding**: When sending data, the system must:
     1. Serialize the data (e.g., using Protobuf).
     2. Calculate and append the CRC.
     3. Construct the header fields (including length, flags).
     4. Output the final byte stream to the hardware layer.

- **Decoding**: On reception, the system must:
    1. Read the header and extract length, CRC, and any additional fields.
    2. Verify that the received payload length matches the header.
    3. Perform a CRC check on the payload.
    4. If valid, pass the payload to the Protobuf decode function; if invalid, handle errors.

4. **Optional Security Features**
    - You may include an **encryption flag** in the header to indicate that the payload is encrypted.
    - If using a simple cipher or an HMAC for authentication, detail how keys are stored and managed.
    - This step is not strictly required but can be a bonus for advanced students.

Design Note: Document how your header is laid out in memory. For example:

Byte Offset	**Field**	**Description**
0	Start-of-Packet	Optional (e.g., 0x7E)
1–2	Packet Length	16-bit length field (little-endian)
3–4	CRC (16-bit)	CRC-16 result of payload bytes
5	Flags/Key	Optional security or type info
6+	Payload	Encoded data (Protobuf)

2.4 Application Layer

The Application Layer provides **user-facing APIs** and leverages the Message Processing Layer for sending and receiving structured data.

1. **High-Level APIs**
    - Implement easy-to-use functions, for instance:

```
bool bkit_send_message(const sensor_data_t* data);
bool bkit_receive_message(sensor_data_t* data);
```

- Internally, these functions should:
    1. Invoke Protobuf encode/decode routines.
    2. Attach or parse the BKIT-COM header.
    3. Invoke the hardware driver functions to send or receive raw bytes.

2. **Data Structures**
    - Decide on what data structures you are passing around. For example, a `sensor_data_t` struct containing fields like `sensor_id`, `value`, and `timestamp`.
    - Provide clear definitions in a header file and describe how these correspond to the Protobuf schema (see Section 2.5).
3. **Error Handling & Logging**
    - If a message fails CRC checks or other validations, the Application Layer should return an error code or handle it gracefully.
    - Consider how to handle partial packets or unexpected timeouts (especially over unreliable connections).
    - (Optional) Include **debug logging** at various levels to help diagnose issues (e.g., print “Invalid CRC” or “Received partial packet of length X”).
4. **Extensibility**
    - The Application Layer should be flexible enough that new message types or fields can be introduced with minimal changes.
    - Use a version number in the header or maintain backward compatibility by carefully adding fields to your Protobuf schema (backwards-compatible changes are possible if done correctly).

2.5 Use of Protobuf and nanopb

Protobuf (Protocol Buffers) is a language-neutral, platform-neutral way of serializing structured data. **nanopb** is a particularly small implementation of Protobuf in C, making it well-suited for embedded systems.

1. **Defining Your Protobuf Schema**
    - Create a `.proto` file that describes the messages you plan to send. For example:

```
syntax = "proto3";
message SensorData {
  uint32 sensor_id = 1;
  float sensor_value = 2;
  uint64 timestamp = 3;
}
```

    - Include a `message` definition for each data type you intend to transmit.
2. **nanopb Generation**
    - Install or configure **nanopb** in your build environment.
    - Use `protoc` with the nanopb plugin to generate `.c` and `.h` files (e.g., `SensorData.pb.c` and `SensorData.pb.h`).

    - Document the steps to run the generator and any relevant build flags or environment variables.
3. **Integration**
    - In your **Message Processing Layer**, call `pb_encode()` to serialize a `SensorData` structure into a buffer. Then attach the BKIT-COM header and send it out.
    - Upon reception, extract the payload (after verifying length and CRC), and call `pb_decode()` to reconstruct the `SensorData` structure.
    - Make sure to handle **buffer size** constraints carefully. nanopb typically uses static buffers, so choose a sufficient buffer size for your largest expected message.
4. **Testing & Validation**
    - Consider creating a simple test harness that:
        1. Builds a `SensorData` struct in code (e.g., `sensor_id=5, sensor_value=24.7, timestamp=123456789`).
        2. Encodes it using Protobuf (with nanopb).
        3. Generates a BKIT-COM packet with header and CRC.
        4. Decodes the same packet on the receive side, verifying that all fields match the original data.
    - If you have multiple embedded boards, test sending data from one board to another, verifying correctness on both sides.

Putting It All Together

By satisfying these **detailed requirements**, your BKIT-COM protocol will feature:

- **Interface independence**: Easily switch between UART, SPI, or I2C without changing higher-level logic.
- **Robust framing**: Consistent packet headers, CRC checks, and optional security flags.
- **Protobuf-based serialization**: Cross-platform, extensible data structures via nanopb.
- **Clear separation of concerns**: Each layer (hardware, message processing, application) handles its own responsibilities, promoting maintainability and clarity.

---

3. Deliverables

1. **Design Document**
    - A short report (2–4 pages) outlining:
        - Chosen hardware interface(s) and why.
        - Header format (with diagrams).
        - CRC details.
        - Protobuf schema definition and workflow.
    - Cite references for interface specifications (UART, SPI, I2C) and any cryptography libraries if used.
2. **Source Code**
    - Well-organized code demonstrating:

        - Hardware interface driver(s).
        - Message processing (header creation, CRC calculation, Protobuf encode/decode).
        - High-level APIs (send/receive).
    - Include a **README** with build instructions.
3. **Demonstration**
    - Short video or live demonstration (if applicable) showing:
        - System sending a Protobuf-encoded packet with the custom BKIT-COM header.
        - System receiving and decoding the packet.
        - (Optional) Multi-interface switching demonstration.
4. **Test Results**
    - Provide sample output (hex dumps, console logs, or screenshots) that illustrate:
        - Proper handling of valid packets.
        - Detection of invalid CRC / corrupted packets.
        - Decoded Protobuf fields on the receiver side.

---

4. Evaluation Criteria

Criterion	Weight	Description
Protocol Design & Layering	20%	Properly implemented 2–3 protocol layers (hardware, message processing, application). Clarity in header design and layering approach.
Implementation & Code Quality	30%	Clean, modular, and well-documented source code. Effective usage of nanopb for Protobuf integration.
Functionality & Testing	30%	Successful demonstration of sending/receiving messages, CRC validation, and correct Protobuf serialization/deserialization.
Documentation & Clarity	20%	Clear design document, references, and usage instructions. Proper citations to protocol references, hardware datasheets, Protobuf docs, etc.

---

5. References

- **Protobuf (Protocol Buffers) Documentation**: https://developers.google.com/protocol-buffers/docs/overview
- **nanopb: Small Protobuf Implementation in C**: https://jpa.kapsi.fi/nanopb/